
Algorithm 12 Validate and Check RDF URIs Using Groq LLM

```
1: Load RDF graph from file
2: Define property_prompts as a dictionary mapping property names to
   prompt texts
   is_uri_validuri
3: Send HTTP HEAD request to uri
4: if response is successful then
5:   return True
6: else
7:   return False
8: end if
   extract_uris_by_propertygraph
9: Initialize uris_by_predicate as an empty dictionary
10: for each triple (subject, predicate, object) in graph do
11:   if object is a URI then
12:     Extract property name from predicate
13:     Add object URI to uris_by_predicate[property]
14:   end if
15: end for
16: return uris_by_predicate
   check_relevance_with_aiuri, prompt_text
17: Extract entity name from uri
18: Build LLM prompt using entity and prompt_text
19: Call Groq-compatible LLM with the prompt
20: if response starts with "true" then
21:   return True
22: else
23:   return False
24: end if
   validate_and_check_allgraph
25: uris_by_property  $\leftarrow$  extract_uris_by_propertygraph
26: Initialize total_counts as empty dictionary
27: for each property, uris in uris_by_property do
28:   Get prompt_text for property from property_prompts
29:   Set valid_and_relevant_count to 0
30:   for each uri in uris do
31:     valid  $\leftarrow$  is_uri_validuri
32:     relevant  $\leftarrow$  check_relevance_with_aiuri, prompt_text
33:     if valid and relevant then
34:       Increment valid_and_relevant_count
35:     end if
36:     Wait for 5 seconds
37:   end for
38:   Store valid_and_relevant_count in total_counts[property]
39: end for
40: Print final summary of valid and relevant URIs per property
41: Execute validate_and_check_allgraph
```
